MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

# RAPORT

## Lucrarea de laborator nr. 3 · Varianta 1

## Disciplina: Bazele Programării în Java

## Tema: Instrucțiuni repetitive (cicluri)

## Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

# Scopul și sarcinile lucrării

Calcularea sumei numerelor pare de la 2 la n, utilizând instrucțiunea for.

# Material teoretic

Instrucțiunea for grupează inițializarea, condiția de continuare și actualizarea contorului. Corpul ciclului se execută cât timp condiția este adevărată. Pentru parcurgerea numerelor pare, contorul pornește de la 2 și crește cu 2. Astfel, nu mai este necesar un test de paritate în interiorul ciclului.

Variabila acumulatoare conține suma parțială. Valoarea ei inițială este zero, elementul neutru al adunării. După fiecare iterație, acumulatorul este suma tuturor numerelor pare deja parcurse. Dacă n este 0 sau 1, corpul nu se execută și suma mulțimii vide este zero.

Numerele naturale impare nu se adaugă, dar n poate fi impar: pentru n = 9 se adună 2 + 4 + 6 + 8. Programul respinge valorile negative, deoarece interfața definește n ca întreg nenegativ. Această convenție elimină ambiguitatea unui interval cu capăt superior negativ.

BigInteger reprezintă întregi cu precizie arbitrară. Metoda add întoarce un obiect nou, deci rezultatul trebuie atribuit din nou variabilei suma. Utilizarea sa evită depășirea tipului int la însumare. Limita de un milion pentru n este o limită practică de timp, declarată în mesajul de intrare. Algoritmul rămâne unul iterativ, conform cerinței, chiar dacă există și o formulă directă de verificare.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

## Algoritmul și verificarea rezultatului

Algoritm: se citește n, se validează intervalul, se inițializează suma cu zero și se adună 2, 4, 6, … până la n inclusiv.

Dacă k = ⌊n / 2⌋, atunci S = 2(1 + 2 + … + k) = k(k + 1). Formula este utilizată ca verificare independentă, nu în locul ciclului.

Pentru n = 10, k = 5 și S = 5 · 6 = 30. Pentru n = 1 rezultă zero. Pentru n = 1000000, S = 500000 · 500001 = 250000500000, o valoare care depășește int.

Ciclul efectuează ⌊n/2⌋ iterații. În limitele declarate, costul este O(n) operații de acumulare; BigInteger ocupă spațiu proporțional cu numărul de biți ai sumei.

## Planul de testare și rezultatele

Cazul 1: 10. Verificare: „= 30”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: 1. Verificare: „= 0”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: -4. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 1000000. Verificare: „= 250000500000”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

# Codul integral — Lab03.java

```
import java.math.BigInteger;
import java.util.Scanner;

public class Lab03 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 3, varianta 1");
        System.out.println("Introduceți n întreg nenegativ (maximum 1000000):");
        if (!in.hasNextInt()) {
            System.out.println("Eroare: n trebuie să fie întreg.");
            return;
        }
        int n = in.nextInt();
        if (n < 0 || n > 1000000) {
            System.out.println("Eroare: n trebuie să aparțină intervalului [0, 1000000].");
            return;
        }
        BigInteger suma = BigInteger.ZERO;
        for (int i = 2; i <= n; i += 2) {
            suma = suma.add(BigInteger.valueOf(i));
        }
        System.out.println("Suma numerelor pare = " + suma);
    }
}
```

# Capturi de ecran și interpretare

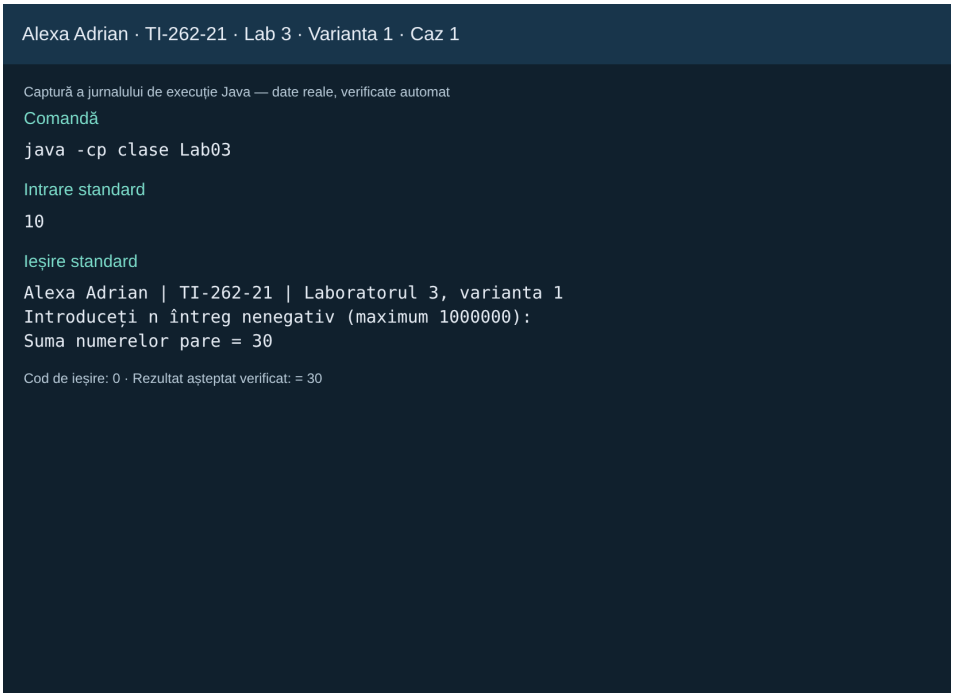



Figura 1. Cazul 1: rezultatul așteptat „= 30” a fost confirmat de execuție.

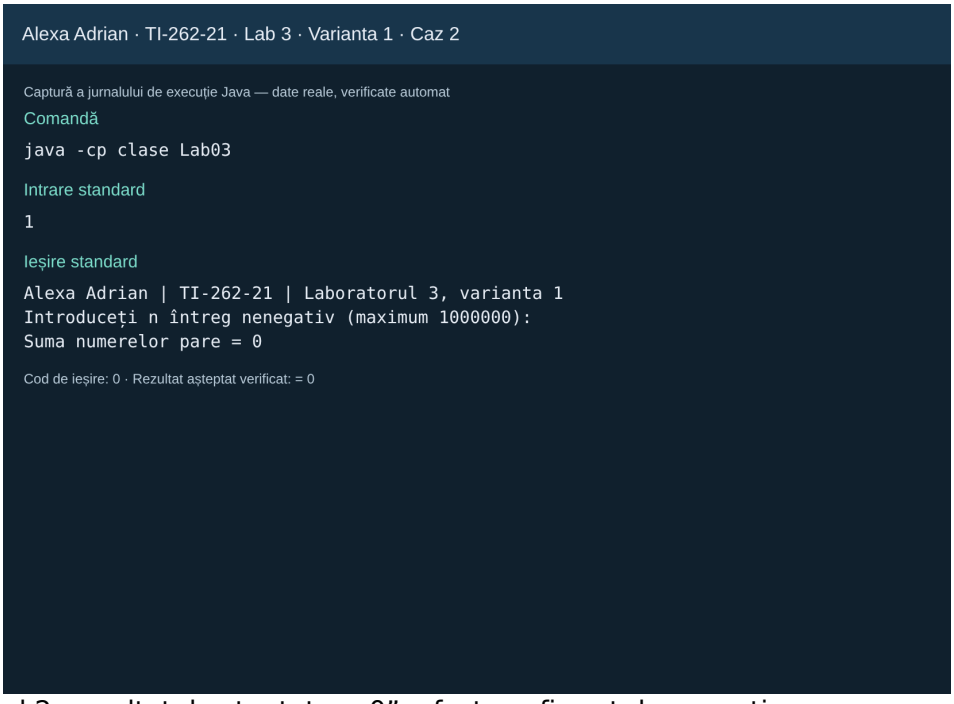



Figura 2. Cazul 2: rezultatul așteptat „= 0” a fost confirmat de execuție.

## Capturi de ecran și interpretare




Figura 3. Cazul 3: rezultatul așteptat „Eroare” a fost confirmat de execuție.

## Concluzii

1. Pasul 2 parcurge exclusiv numerele pare și simplifică bucla.

2. Formula k(k + 1) confirmă independent suma obținută iterativ.

3. BigInteger previne depășirea numerică observabilă pentru valori mari ale lui n.

## Întrebări pentru pregătirea susținerii

• De ce suma inițială este zero?

• Ce rezultat avem pentru n = 1?

• De ce reatribuim suma.add(...) ?

## Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab03.java, apoi java -cp clase Lab03. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

## Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.